

Re-Meltrix: A Reconfigurable Processing-in-Memory Architecture Based on RRAM and Function Synthesis

Boyu Long¹, Yinhe Han¹, *Senior Member, IEEE*, Xian-He Sun², *Life Fellow, IEEE*,
and Xiaoming Chen¹, *Member, IEEE*

Abstract—The reconfigurable processing-in-memory (PIM) architecture has garnered significant attention in recent years due to its versatility and ability to overcome storage limitations. However, it faces challenges, such as overly complex mapping and routing caused by the fine granularity of basic logic units, and the inclusion of numerous redundant devices to achieve reconfigurability. To address these issues, we have designed a software–hardware co-design reconfigurable PIM architecture called Re-Meltrix. Its hardware architecture uses an resistive random-access memory array as the foundation, combined with well-designed peripheral circuits. Maintaining a controllable area, it integrates logic, storage, ternary content-address memory, and interconnection modes into a unified tile architecture and implements two-level independent interconnection within and between tiles. This approach achieves a single tile logic capacity multiple times that of the most advanced reconfigurable PIM architectures currently available, thereby resolving mapping and routing difficulties at the hardware level. Our proposed function synthesis, combined with the hardware architecture, specifically optimizes two-level interconnection separation and module segmentation, further reducing interconnection complexity and improving tile usage efficiency. Experiments have demonstrated that our architecture outperforms the state-of-the-art Liquid Silicon by 2.00–4.31× in performance and reduces power consumption by 29%–68%. Compared with the previously published Meltrix, the area has decreased by 15%–35%, with the area and power consumption remaining almost unchanged.

Index Terms—Processing in memory, resistive random-access memory (RRAM), software–hardware co-design, ternary content-addressable memory (TCAM).

I. INTRODUCTION

PROCESSING-IN-MEMORY (PIM) is a key solution for addressing the memory wall challenge that is caused by the uneven performance development of processors and

storage devices. Its emergence has enhanced the performance and energy efficiency in applications like deep neural networks (DNNs) [1], [2], [3], [4], [5], ternary content-addressable memory (TCAM)-based content query systems [6], [7], [8], and other memory-intensive tasks [9], [10], [11]. However, most existing PIM architectures are focused on accelerating a single class of applications. The increased binding to specific applications limits the design flexibility and applicability of PIM architectures. For multifunctional and evolving applications, single-application architectures lack development potential.

To address the limitations of single-purpose PIM architectures in terms of generality, reconfigurable PIM architectures [12], [13], [14] have been designed. These architectures use traditional reconfigurable frameworks [e.g., field programmable gate arrays (FPGAs)] as guidance, replace static random-access memories (SRAMs) with popular nonvolatile devices (e.g., resistive random-access memories (RRAMs) [15], [16], [17], [18]), and focus on the combination of storage modules [e.g., embedded memory blocks (EMBs)] and logic modules [e.g., lookup tables (LUTs)]. These designs allow the architectures to have an underlying hardware structure that can implement storage and logic functions separately in different situations. They inherit reconfigurability and break through the memory wall by reducing data handling processes. Simultaneously, the integration of dynamic random-access memory (DRAM) with computing modules (LUT-based PIM architectures [19], [20]) has been extensively studied, as DRAM remains one of the most widely used storage devices. Such architectures typically incorporate logic components at various levels of the DRAM hierarchy, connecting them to the read path to enable data retrieval while performing computations.

Although these architectures combine the advantages of in-memory processing and programmability, they also inherit serious issues from both. First, in general logic implementations, in-memory processing devices often use the bistable state to simulate switches, thus achieving logic gate functions [12], [13], [14]. This underlying implementation method requires applications to map logic gate operations as a foundation, and then map them to the architecture. However, such low-level implementation methods, when combined with general programmable architectures, cause simple functions to occupy excessive resources, making it unsuitable for deploying

Received 7 September 2024; revised 25 December 2024; accepted 2 February 2025. Date of publication 12 February 2025; date of current version 22 August 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62122076, Grant 62025404, Grant 62488101, and Grant 62495104; and in part by the Youth Innovation Promotion Association CAS. This article was recommended by Associate Editor A. Gamatié. (*Corresponding author: Xiaoming Chen.*)

Boyu Long, Yinhe Han, and Xiaoming Chen are with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China, and also with the School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing 100190, China (e-mail: longboyu21b@ict.ac.cn; yinhes@ict.ac.cn; chenxiaoming@ict.ac.cn).

Xian-He Sun is with the Department of Computer Science, Illinois Institute of Technology, Chicago, IL 60616 USA (e-mail: sun@iit.edu).

Digital Object Identifier 10.1109/TCAD.2025.3541487

large-scale applications. Second, these logic implementation methods, along with the rapidly increasing demand for logic quantity and complexity, place significant pressure on routing [21]. In DRAM-based architectures, the placement of logic devices and data paths is fixed, limiting them to performing the same operations on data at a specific level. Consequently, when addressing diverse application types, a single architecture often fails to meet the requirements of all applications.

Liquid-Silicon (L-Si) [22] is a classic reconfigurable PIM architecture. It uses RRAMs as the basic units and designs storage-logic integrated arrays. An RRAM device can function as either a logic gate or a 1-bit storage unit. Based on this mechanism, each array in L-Si can be configured to perform computation, storage, or interconnection. To address the routing overhead problem of the architectures mentioned above, it attempts to reduce the depth of the overall logic network through multi-input (~ 30 inputs) sum-of-product (SOP) logic, thereby indirectly reducing the logic complexity of applications and enabling them to map to an acceptable number of hardware resources. It also posits that reducing the logic network depth can decrease signal transmission between hardware resources, thus reducing routing pressure. However, this method overlooks the issue of input signal duplication during the implementation of SOP. Although it reduces the depth of the logic network, the large number of signal duplications introduced by SOP does not reduce the routing proportion.

Unlike L-Si, our previous work on Meltrix [23] achieves a novel reconfigurable PIM architecture based on homogeneous RRAM arrays by combining logic and storage patterns using LUTs. The RRAMs in an array can serve as a truth table for LUTs or as storage units. Its logic mode based on LUTs enables it to support single-module logic capacity several times that of L-Si, while achieving multilevel lossless interconnection within a single module. This design increases the granularity of logic modules and avoids significant signal duplication, effectively solving the routing overhead problem present in L-Si. Additionally, it can achieve the same distributed storage function as L-Si, and realize low-power read/write address decoding peripheral circuits, reducing the logic requirements of the storage module by 67%, with lower power consumption and higher flexibility than L-Si.

However, in Meltrix, to achieve multilevel lossless interconnection within a single module, we use numerous selectors and extensively replicated the inputs and outputs. Although this replication is confined within the module and does not impact interconnection, it significantly increases the tile area, making Meltrix's tile area nearly $2.5\times$ larger than that of L-Si. Additionally, Meltrix does not support TCAM, rendering it weak in information search and matching applications. To address the aforementioned issues, in this article, we extend Meltrix to develop Re-Meltrix. This article extends from [23] in the following aspects. Re-Meltrix implements hardware interconnection restructuring and TCAM support, thereby resolving Meltrix's large single-tile area and lack of data search and matching capabilities. Additionally, the core clustering algorithm has been optimized in software, and a

custom mapping process has been added, further enhancing overall performance and flexibility.

Specifically, Re-Meltrix is composed of a number of homogeneous arrays that employ RRAM arrays as the basic structure, combined with peripheral circuits to enable flexible reconfiguration into logic, storage, interconnection, and TCAM modes. It utilizes LUT input replanning, inheriting a logic pattern with a single module logic capacity multiple times larger than that of L-Si, while significantly reducing resource and area usage. This implementation has minimal impact on the multilevel lossless interconnection within a single module, effectively addressing the routing overhead problem of L-Si and the area problem of Meltrix. Additionally, Re-Meltrix supports TCAM mode without adding any new hardware, resulting in significant improvements in performance, power consumption, and area, compared with L-Si.

In terms of software, we have enhanced the two-stage top-down minimum communication clustering (TTMCC) method based on the Meltrix software workflow, resulting in improved clustering performance and speed. Additionally, we have implemented custom recognition and mapping of modules by adding recognition keywords to the module in the hardware description, allowing for the separation of modules from the software process and enabling user-defined mapping. We have built in two custom mappings: 1) storage module and 2) TCAM module. Users can add other modules themselves, such as multipliers for specialized optimization. This software process, combined with the improved hardware architecture, minimizes the complexity of global routing while enhancing the flexibility and configurability of mapping, thereby reducing mapping latency and power consumption.

In summary, we have made the following contributions in this article.

- 1) We propose a multifunctional reprogrammable RRAM-based architecture called Re-Meltrix, which supports coarse-grained logic patterns, storage patterns with simplified logic, interconnection patterns, and TCAM patterns for content matching. By leveraging the commonalities of these modes, Re-Meltrix is designed to support all four functions within a single tile structure by reusing various paths in the array and peripheral circuits.
- 2) We propose a function synthesis process compatible with the Re-Meltrix hardware architecture, including a new fully interconnected basic unit (FIBU) unit definition, retro-TTMCC (RTTMCC) algorithm, and custom module identification and separation. The first two provide excellent logic optimization and layout performance, while the latter greatly enhances the customization and configurability of the entire process.
- 3) We conduct detailed experiments on Re-Meltrix from the tile level to the application level. We use L-Si and Meltrix as the baselines to compare the single tile area of Re-Meltrix and the focused applications of logic, memory, and TCAM modes. A detailed analysis is conducted on these results.

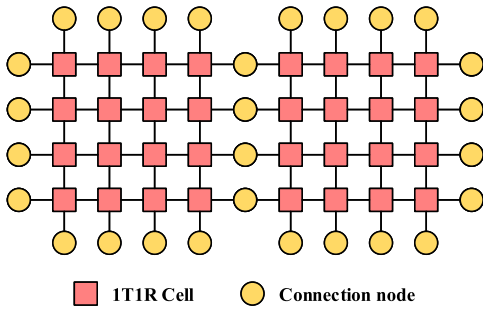


Fig. 1. Liquid-silicon architecture.

II. BACKGROUND

A. Resistive Random-Access Memory

The RRAM, as one of the mainstream nonvolatile memory devices, offers advantages, such as excellent scalability [24], full compatibility with complementary metal-oxide-semiconductor (CMOS), fast read and write speeds [25], and low operating voltage. By applying an appropriate voltage across an RRAM, its resistance value can be modified. Additionally, when a weak input voltage is applied, the resistance state of an RRAM can be measured without altering its resistance value. This straightforward read–write method enables the construction of high-density and low-power crossbars.

More importantly, due to its bistability, an RRAM can function both as a switching device (high resistance state (HRS) for state *off*, and low resistance state (LRS) for state *on*) and as a storage device (bistability can store 1-bit information) [26], [27]. These characteristics enable RRAMs to perform functions that would typically require multiple transistors, making it a potential foundational component for reconfigurable architectures.

B. Related Works

Several attempts to combine RRAMs with reconfigurable architectures have been published prior to our work. However, most of these efforts use RRAMs as an enhanced component within existing reconfigurable architectures, such as FPGAs, by replacing interconnection blocks [16] and switching blocks or constructing high-density EMBs and TCAM [17]. These improvements address only specific aspects and fail to resolve the inherent routing overhead problem in reconfigurable architectures. To address this issue, L-Si attempted to construct a crossbar composed of RRAMs, as shown in Fig. 1. Nonetheless, the structure, comprising crossbars and simple peripheral circuits, falls short of solving the problem. For instance, when operating in storage mode, three adjacent tiles of a memory tile must be configured as related logic decoding circuits, leading to significant resource waste. Furthermore, using SOPs instead of LUTs for logic introduces numerous signal duplicates, which increase routing complexity. L-Si lacks internal routing, causing all routing to be handled by the interconnection module. Since the interconnection module and the regular logic module share the same tile design, the

high cost of using each interconnection module exacerbates the resource usage for routing.

III. RE-MELTRIX HARDWARE ARCHITECTURE

A. Overview

In this section, we provide a detailed introduction to the hardware architecture of Re-Meltrix, which is achieved by tiling a single, uniform tile. The basic unit consists of an RRAM array and peripheral circuits. Fig. 2(a)–(d) illustrate the top-level hardware structure of Re-Meltrix, where each tile is directly connected to the adjacent four tiles via signal lines. Specifically, the data output (DOUT) of each tile drives inputs, including data input (DIN), write enable (WEN), column address (Col Addr), and row address (Row Addr), in four connection directions, while each tile receives inputs from four directions. Each tile can be configured into four distinct modes: 1) logic mode; 2) interconnect mode; 3) storage mode; and 4) TCAM mode. This flexibility allows us to adjust the ratio of storage, TCAM, and computation, as well as the position of each module, according to various applications. These advantages significantly reduce resource constraints and global routing challenges caused by the fixed positions and quantities of nonlogic devices, such as storage and TCAM.

Fig. 2(b) illustrates the construction of the RRAM array and peripheral circuits within the tile. The tile primarily consists of a 2-D RRAM unit (RU) array, a row decoder, a column decoder, an LUT line (LL) control router, configurable bidirectional RRAM selectors, and output amplifiers. The RU, which serves as the foundation of the multimode architecture, will be detailed in the next paragraph. The row and column decoders comprise masking circuits and one-hot decoders which converting input row and column signals into activation levels corresponding to addresses, while the masking circuits support storage modes of varying word lengths. Output amplifiers perform different functions by configuring various input signal lines, reference levels, and output branches, with specific paths explained alongside the introduction of different modes. It is important to note that for an $N \times N$ RU array, there are N output generators (OGs) corresponding to it. These N OGs correspond to the array in two ways: the m th OG corresponds to all RUs in the m th row, or the m th OG corresponds to all RUs in the m th column. These two correspondence methods involve different signal line connections, which will be elaborated upon in the next paragraph.

An RU is a one-transistor-one-RRAM (1T1R) unit composed of one RRAM and one transistor, featuring four ports: 1) word line (WL); 2) bit line (BL); 3) sense line (SL); and 4) LL. The WL controls the on/off state of the transistor, with its two ends connected to the output port of the row decoder and the gate of the transistor, respectively. The BL is responsible for concatenating the output of the row decoder, all transistors in the corresponding column, and the OG of the corresponding column. The SL is connected to the RRAM and an RRAM-multiplexer (RMUX), which selectively connects to DIN or ground. The LL is led out from the connection port between the transistor and RRAM and connected to the OG of the corresponding row. These four ports form two different paths.

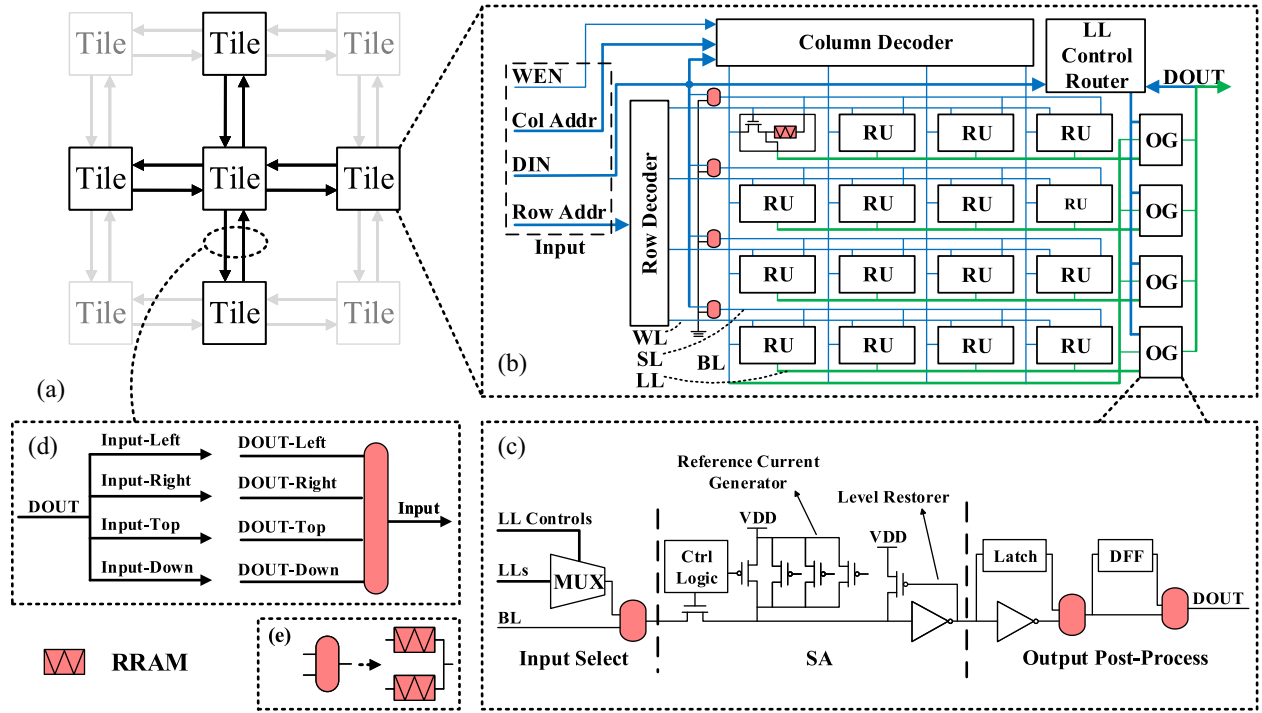


Fig. 2. Re-Meltrix architecture. (a) Tile connection of Re-Meltrix. (b) Tile architecture of Re-Meltrix. (c) OG (d) Detailed structure of input and output of a tile. (e) RRAM-MUX.

- 1) When the transistor is turned on by the WL, the BL and SL form a WL-RRAM-transistor-BL path, which is the same as the read and write path of standard storage devices. The BL is then connected to the OG of the corresponding column.
- 2) When the transistor is turned off by the WL, the SL and LL bypass the transistor and form the SL-RRAM-LL path. At this point, all LLs in each row will be independently connected to the OG of the corresponding row.

This configurable multichannel feature, combined with the configurable selective input of the OG, supports the implementation of multimode functions.

Fig. 2(e) illustrates the specific details of the RMUX. Generally, an RMUX with $N + 1$ ports consists of N RRAMs, with one end of all RRAMs connected together and the other ends serving as independent ports. By adjusting the resistance value of the RRAM in the RMUX, it can function as a selector or fanout, depending on the input and output orientation defined by the module. It is important to note that the resistance setting of the RRAM in the RMUX is configured during the initialization phase, which coincides with the burning of the LUT content, and does not change during normal operation. Furthermore, all RRAMs discussed in this article use binary models, specifically HRS and LRS. Therefore, the RMUX can form multiple different paths as needed during the initialization phase, significantly supporting multimode functionality.

The details of OG are shown in Fig. 2(c). OG consists of three parts: 1) the input selection; 2) the sense amplifier (SA); and 3) the output post-processing. The input selection can accept the corresponding column BL or all corresponding

rows as input. When selecting the latter as input, the DIN and DOUT within the tile jointly drive the LL control router, generating the corresponding LL control signal for the OG. The LL control signal serves as the selection signal for OG input, controlling the selection of the corresponding LL to enter the SA. The selection of BL and LL, as well as the selection of control signals for the LL control router, is implemented by RMUX, achieving a high degree of flexibility. The SA consists of a reference current generator and a level restorer. The reference current generator selects the appropriate reference current for the input charging and discharging process based on the tile mode. The level restorer compensates for the level loss when the output signal passes through the front-end input selector and current reference generator, ensuring the standardization of the output signal. It should be noted that VDD in Fig. 2(c) refers to the power supply voltage. The output post-processing section is composed of latch/inverter branches and flip-flops/through branches connected in series, responsible for output holding and timing signal generation. According to the tile's mode and output requirements, flexible choices can be made by configuring RMUX.

B. Logic Mode

In logic mode, each row (the RUs in row and corresponding OG) in the tile forms an LUT, with the RUs storing the truth table and the OG generating the output. Thus, a tile containing $N \times N$ RUs can form up to $N \log(N)$ -input LUTs. The specific signal pathways are shown in Fig. 3(a), where the red parts represent data pathways, the blue parts represent control signals, and the gray parts are inactive. In this mode,

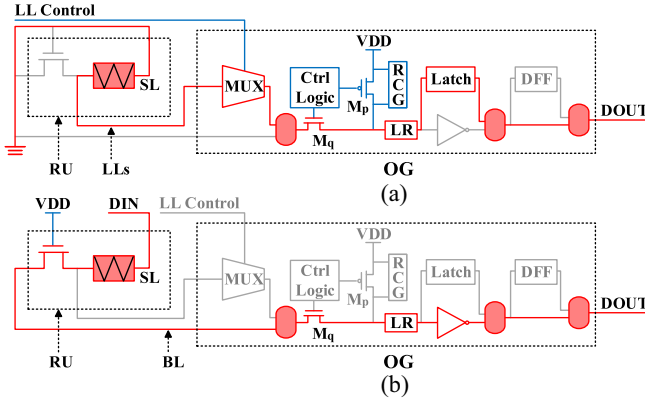


Fig. 3. (a) Data path in *logic* mode. (b) Data path in *interconnection* mode.

SL, WL, and BL are all grounded, and OG selects the LL selection signal generated by LL control router and all LL in the corresponding row as input. The inverter branch and data flip-flop (DFF) branch are bypassed, resulting in a combined output signal. If necessary, the direct branch can be bypassed, and the DFF branch can be selected. This yields an LUT that outputs timing signals, which is particularly suitable when the LUT output is closely followed by a latch.

The operation of the logic mode is as follows. During the precharging stage (when the clock signal is low), transistor M_p is turned on, and transistor M_q is turned off, allowing the input of the level restorer to be directly connected to VDD, while the latch is in a hold state, making the output unaffected by the level restorer. In the evaluation stage (when the clock signal is high), M_p is turned off, and M_q is turned on, allowing the level restorer to connect to the selected RU through the LL. If the selected RU is in the LRS, the input of the level restorer will be grounded through SL, causing its level to quickly decrease and the level restorer to output a high level. Conversely, if the selected RU is in the HRS, the level restorer will be almost disconnected from the ground, and its input will not undergo significant changes, keeping the output at a low level. At this point, the DIN and output are connected to the OG through an LL control router, and the selected RU's resistance state is measured through charging and discharging to achieve the LUT lookup process.

To provide normal query and feedback connection functions for the LUT, we chose DIN and DOUT as the selector for the LUT at the software level. However, we observed that a large amount of DIN and output was duplicated at the OG, significantly increasing the number of RMUX and leading to wastage of area. Therefore, we introduced an LL control router that indirectly generates LL control signals using DIN and output. The input of each LUT will no longer use all the DIN and output in this tile but will be divided into multiple groups, as shown in Fig. 4.

For tiles containing an $N \times N$ RU array, both the DIN and output have N bits. We divide the DIN and output into $m + 1$ parts, with the last part occupying M bits and the remaining $N - M$ bits evenly divided among the first m parts. Simultaneously, we evenly divide the OG into m parts ($N = 64$, $M = 2$, and $m = 2$ in Fig. 4). Following this division, the

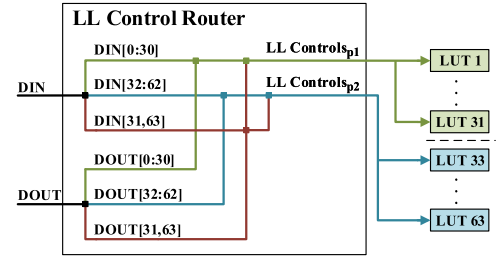


Fig. 4. LL control router.

mapping relationship between DIN and output and OG is as follows.

- 1) The first m parts of DIN and output correspond one-to-one with the m parts of OG, with their DIN and output interconnected within this section.
- 2) The $(m + 1)$ th part of DIN and output will connect to all OGs.

This partitioning method greatly reduces RMUX consumption while maximizing the internal interconnectivity of tiles. Specifically, the relationship between the output bit width of the LL control router and the parameters M and m is as follows:

$$\text{Width}_{\text{Output}} = \frac{2(N - M)}{m} + 2M. \quad (1)$$

In the example shown in Fig. 4, LL controls only require 66 bits, while the total DIN and output width is 128 bits. Maintaining the interconnectivity within the tile transforms the global routing from fine-grained LUT-LUT connections to coarse-grained tile connections, thereby concealing many intricate interconnections within the tile and significantly reducing the complexity of global routing. It is important to note that the parameters M and m of the LL control router are hardware-fixed, similar to the array size, and cannot be reconfigured. Therefore, achieving a balance between reducing area and enhancing interconnectivity is essential. This issue will be discussed in conjunction with the experimental results in Section V-A.

C. Interconnection Mode

The interconnection mode allows mapping inputs to any position and number of outputs. The specific signal pathway is shown in Fig. 3(b), where the red part represents the data pathway, the blue part represents the control signal, and the gray part indicates nonfunctional elements. In this mode, all WLs are set high, activating all transistors in the RU array. SL is connected to the DIN, and OG selects the corresponding column's BL as the output. The clock logic in SA is turned off, M_q remains normally open, and Latch and DFF are bypassed, allowing the entire OG to function as a level restorer. To map the m th bit of the DIN to the n th bit of the DOUT, the RRAM in the m th row and n th column must be set to LRS, while the remaining RRAMs in the n th column are set to HRS. This configuration allows the m th bit of the DIN to pass through the LRS RRAM to reach the OG, while the HRS RRAMs in that column will influence the m th bit's signal with other resistance

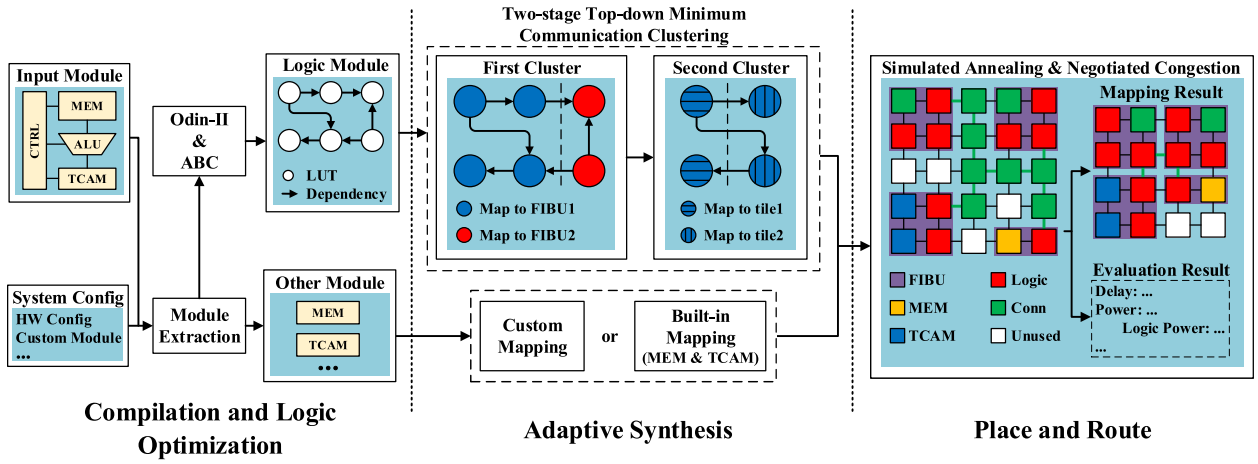


Fig. 7. Function synthesis framework of Re-Meltrix.

realize its potential, we have developed a comprehensive function synthesis methodology to optimize the hardware mapping process.

As shown in Fig. 7, this process utilizes hardware description language (HDL) [28], [29] as the model input, along with other necessary inputs, such as hardware configuration information and custom module layouts. The first step is module extraction. The top-level module separates custom modules and storage-related modules, leaving pure logic circuits for further optimization. In this step, users can freely peel off modules that require customization or self-optimization, providing ample flexibility and configurable space for the comprehensive process. The stripped pure logic circuit is then optimized by ODIN-II [30] (derived from the popular open-source FPGA synthesis tool VTR [31]) and ABC [32] (an open-source logic synthesis optimization tool), and transformed into the k-input-LUT form described by Berkeley logic interchange format (BLIF) [32].

The second step is the adaptive synthesis process. In this step, the optimized logic netlist is transformed from a BLIF representation to a directed acyclic graph (DAG) representation, where vertices represent LUTs and edges represent dependency relationships. Following this, an initial layout suitable for Re-Meltrix is generated using FIBU (refer to Section IV-A) and optimized using RTTMCC (refer to Section IV-B) methods. Additionally, the storage-related modules and custom modules that were stripped in the first part are also mapped to the hardware at this stage. The storage-related modules are built using fixed mapping through function synthesis, while the custom modules are mapped according to the user configuration file. The combination of these two parts completes the initialization layout work of mapping the model to the hardware of Re-Meltrix.

The third step is place and route. This step determines the final position and wiring layout relationship of each tile after initializing the layout. In this step, simulated annealing (SA) [33] is used to explore the design space. SA employs small perturbations to explore the adjacent design space of the current layout, including the movement, exchange, and rotation of FIBUs. After these perturbations, the negotiated

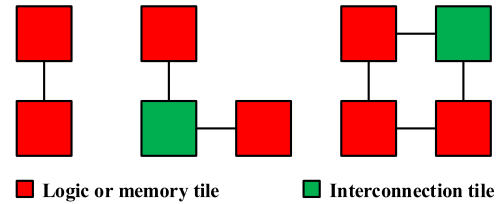


Fig. 8. Examples of FIBU.

consensus (NC) [34] routing algorithm employs a minimum routing negotiation approach to rewire the new layout. Using SA and NC, we aim to minimize routing costs and generate corresponding layout diagrams, critical paths, and other network evaluation results.

A. Fully Interconnected Basic Unit

A FIBU is a collection of tiles where any two tiles must satisfy one of the following rules (examples of different FIBUs are shown in Fig. 8).

- 1) Directly connected (adjacent).
- 2) Connected by a pathway composed of tiles in the interconnection mode of this FIBU.

The characteristic of a FIBU is that any direct dependency relationship with starting and ending points within the same FIBU can be resolved internally without relying on external resources. Therefore, global routing is divided into two parts: 1) interconnection within FIBUs and 2) interconnection between FIBUs. For large-scale applications, this segmentation method can significantly reduce global interconnection pressure. Thus, in the global routing stage, we only optimize the interconnection between FIBUs.

B. Retro Two-Stage Top-Down Minimum Communication Clustering Algorithm

The RTTMCC algorithm proposed in this article is a top-down two-stage clustering algorithm, and its basic clustering algorithm utilizes METIS [35] communication minimization cut clustering algorithm. As shown in Algorithm 1, before applying this algorithm, the pure logic part of the model is

Algorithm 1: Retro TTMCC

Input: Directed LUT graph G
Output: List of FIBUs L_{FIBU} , and list of tiles L_{tiles}

```

1  $FC = \text{MinComPartGraph}_{\text{Metis}}(G)$ 
2  $clusters_{\text{illegal}}, L_{FIBU}, L_{tiles} = \emptyset$ 
3 while  $FC$  is not empty do
4   clear  $clusters_{\text{illegal}}$ 
5   foreach  $cluster$  in  $FC$  do
6     New  $FIBU$ 
7     if  $\text{Mapping}(cluster, FIBU)$  is success then
8       Push  $FIBU$  into  $L_{FIBU}$ 
9     else
10      Push  $cluster$  into  $clusters_{\text{illegal}}$ 
11    Delete  $cluster$  from  $FC$ 
12  Reorganize  $clusters_{\text{illegal}}$  into  $G'$ 
13   $FC = \text{MinComPartGraph}_{\text{Metis}}(G')$ 
14  $FC = L_{FIBU}$ ;
15 while  $FC$  is not empty do
16   clear  $clusters_{\text{illegal}}$ ;
17   foreach  $cluster$  in  $FC$  do
18      $SC = \text{MinComPartGraph}_{\text{Metis}}(cluster)$ ;
19     New  $L_{tile}$ ;
20     if  $\text{Mapping}(SC, L_{tile})$  is success then
21       Push  $L_{tile}$  into  $L_{tiles}$ ;
22     else
23       Push  $cluster$  in  $clusters_{\text{illegal}}$ 
24   Delete  $cluster$  from  $FC$ 
25  Reorganize  $clusters_{\text{illegal}}$  into  $G'$ 
26   $FC = \text{MinComPartGraph}_{\text{Metis}}(G')$ 

```

transformed into a DAG G , where vertices represent LUTs and edges represent the dependency relationships between LUTs. This algorithm uses G as input and outputs a clustering list and tile list that meet the mapping criteria. The pseudocode is shown in Algorithm 1. RTTMCC first uses METIS to perform the initial communication minimization cut on G (first line) and conducts a validity check on each segmented cluster (lines 4–13). Subsequently, the algorithm performs a second communication minimization cut on each cluster, followed by a second validity check (lines 5–26).

The process of the two validity checks is similar. The algorithm checks whether each segmented cluster satisfies the mapping conditions specified for the FIBU or tile. Specifically, it verifies whether the number of cluster inputs and outputs is less than or equal to the number of inputs and outputs in the FIBU or tile, and whether the interconnection within the cluster is within the available interconnection resources in the FIBU or tile (lines 7 and 20). Clusters that meet these conditions are moved to the result list (lines 8 and 21), while those that do not are moved to the illegal cluster list (lines 10 and 23).

Unlike the original TTMCC, the revised algorithm addresses illegal clustering by restoring it to a DAG G' (lines 12 and 25) and performing communication minimization segmentation again. During this phase, segmentation parameters are adjusted, such as increasing the number of clusters or relaxing segmentation conditions. In contrast, the original version splits illegal clusters completely in half, leading to significant

TABLE I
BENCHMARKS AND TEST SAMPLES USED IN EVALUATION

Benchmark	Sample	Description
ISCAS'89	s38417	Large scale sequential circuit
	s38584	
MCNC	dsip	Encryption
	bigkey	
	dalu	Dedicated ALU
	mm30a	Minmax circuit
	mult32a	Multiplier
	mult32b	
VTR	sha	Sha-160 hash
	ch_intrinsics	Memory controller
	mkPktMerge	Memory merge

resource waste, as many illegal clusters exceed the resource limits only marginally, resulting in unused resources in the FIBU or tile. The new RTTMCC algorithm incrementally increases the number of clusters by one until clustering is fully successful, thereby maximizing hardware resource utilization.

Meanwhile, with the addition of LL control routers, conditions for small-scale clustering within tiles are introduced. By dividing the LL control signals within the tile into several segments, a condition analogous to communication minimization clustering is established between these segments. The spontaneous clustering facilitated by the LL control signals, as described above, can classify LUT dependency relationships with greater detail and positively impact the overall layout and routing, as will be demonstrated in the next section.

V. EVALUATIONS

In this section, we conduct a comprehensive performance evaluation of Re-Meltrix. We select the largest combinational logic circuits, namely circuits S38417 and S38584 from ISCAS'89, and test Re-Meltrix with various practical applications from MCNC [36] and VTR [31], including multipliers, arithmetic, and logic units (ALUs), and two complete encryption algorithms. Additionally, we evaluate performance and routing capability with a complete storage control unit and a storage merging unit from VTR to assess its handling of data-intensive applications. Beyond these test samples, we also assess various metrics of the TCAM unit with 1k, 2k, and 4k entries to demonstrate Re-Meltrix superior performance in TCAM mode.

In the experiment, L-Si is used as the baseline. To ensure consistency in experimental conditions, all tests are conducted using 45-nm transistors and RRAMs with identical configurations. The characteristics of RRAM devices used in the evaluation are as follow. 1) $R_{\text{HRS}}/R_{\text{LRS}} = 2 \text{ M}\Omega/4.82 \text{ k}\Omega$, 2) $\pm 1 \text{ V}@10 \text{ ns}$ pulse voltage for set/reset [25], [37]. The power consumption and delay for individual tiles of L-Si, Meltrix, and Re-Meltrix are derived from HSPICE simulations, with schematic diagrams provided by [22] and [23], as well as Section III of this article. Based on these schematic diagrams, the area of a single tile is inferred for future comparisons of the total area. Additionally, L-Si utilizes VTR for timing synthesis and layout routing, thus obtaining the total power consumption and delay for different applications. Meanwhile, Meltrix uses

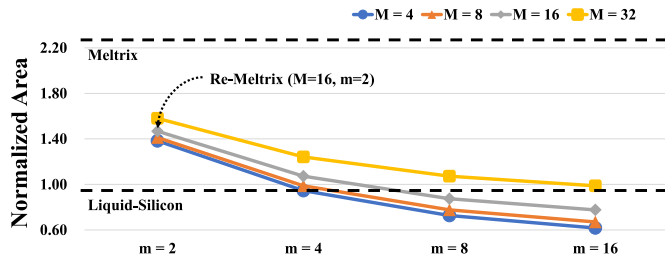


Fig. 9. Area evaluation for different parameters in LL control router.

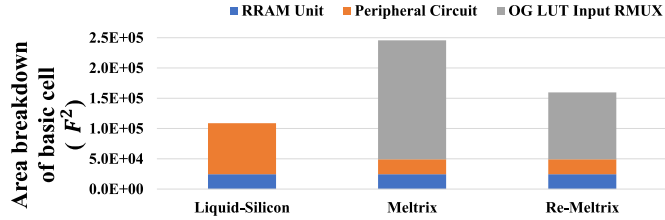


Fig. 10. Area comparison of single tile.

the functional synthesis proposed in [23], and Re-Meltrix employed the software process described in Section IV, which is completed using C++, taking between a few seconds and approximately ten minutes to finish, for optimization, routing, and mapping.

A. Tile Evaluation

Since the parameters M and m of the LL control router significantly influence the tile area of Re-Meltrix, we first calculate the tile area of Re-Meltrix for different values of M and m . The normalized area, relative to L-Si, is shown in Fig. 9, where the two dashed lines represent the normalized areas of L-Si and Meltrix (i.e., 1 and 2.21, respectively). Based on Fig. 9, we can conclude that parameter M has an approximately linear effect on the tile area, while the effect of m is approximately reciprocal. After extensive testing on the test samples in the subsequent sections, we observe that Re-Meltrix exhibits performance identical to Meltrix at $M = 8$ and $m = 2$. Therefore, in the following experiments, we set the LL control router parameters to $M = 8$ and $m = 2$.

Fig. 10 illustrates the area and internal composition of a single tile of L-Si, Meltrix, and Re-Meltrix, with RU array sizes of 64×64 for all three cases. It is evident that L-Si uses almost no RRAM outside the RU array, so the results predominantly reflect the RU array and peripheral circuits. However, due to the architectural design of L-Si, its number of SA units is twice that of Meltrix and Re-Meltrix, and it contains multiple registers for storing configuration information, resulting in a peripheral circuit area more than twice that of Meltrix and Re-Meltrix. For Meltrix, a significant portion of the area is consumed by the LUT input control selector of OG, which accounts for approximately 80% of the total area of a single tile. Building on Meltrix, Re-Meltrix employs LL control routers to achieve early allocation of LUT input control. This approach not only enhances internal interconnection of tiles but also reduces the area by 35% compared to Meltrix, resulting in a single tile area that is

reduced from $2.3\times$ that of L-Si in the Meltrix to $1.5\times$ that of L-Si. Although still larger than single tile area of L-Si, the substantial capacity advantage of LUT implementation logic and the coordination of software architecture ensure that the area of all projects is significantly smaller than L-Si in subsequent testing.

B. Logic Mode Evaluation

All test results for logic mode evaluation are presented in Fig. 11, with all test samples consisting of pure logic circuits. The results include normalized performance (reciprocal of critical path delay), power consumption and area, and the proportion of interconnect power consumption relative to total power consumption. It is important to note that in any architectural comparison experiment, performance, power consumption, and area often exhibit pairwise balanced results. Consequently, this study comprehensively analyzes these three metrics to ensure the accuracy and fairness of the experiment. Furthermore, to maintain fairness, the software process for all test samples adheres to the “optimal performance” standard. The results reveals that Re-Meltrix and Meltrix achieve performance improvements of $1.96\times$ and $1.93\times$ over L-Si, respectively, with power consumption reduced by 59% and 56%, and area reduced by 49% and 62%, respectively. These results underscore the advantages of the software-hardware collaborative architecture proposed in this article. Additionally, Re-Meltrix achieves a 25% reduction in area compared with Meltrix, due to the incorporation of LL control routers, which resulted in a 1% decrease in performance and a 7% increase in power consumption. This indicates that Re-Meltrix accomplishes a 25% reduction in area compared with Meltrix with minimal performance loss and almost no increase in power consumption, demonstrating the effective role of LL control routers. In terms of interconnectivity, Re-Meltrix also demonstrates performance nearly identical to Meltrix, with a 41% reduction in interconnectivity compared with L-Si. Notably, for the *bigkey* example, Re-Meltrix exhibits the same characteristic as Meltrix, with no FIBU external interconnection. This suggests that our LL control router effectively removes redundant interconnections without impacting the internal connectivity of tiles.

C. Memory Mode Evaluation

In this test, we evaluate storage mode performance using L-Si, Meltrix, and Re-Meltrix. Details on the implementation of the storage module can be found in [22], [23], and Section III-D. For testing, we employ storage controllers (e.g., the *ch_intrinsics* sample) and storage merge units (e.g., the *mkPktMerge* sample), each with varying storage capacities.

It should be noted that performance testing of the storage module is not included here because this test primarily focuses on the tile in storage mode which is the main hardware component in the storage module. Consequently, the logic component is relatively simple. In this context, the RRAM read and write times, as discussed in Section III-D, become the dominant factor in module delay, accounting for over 80%

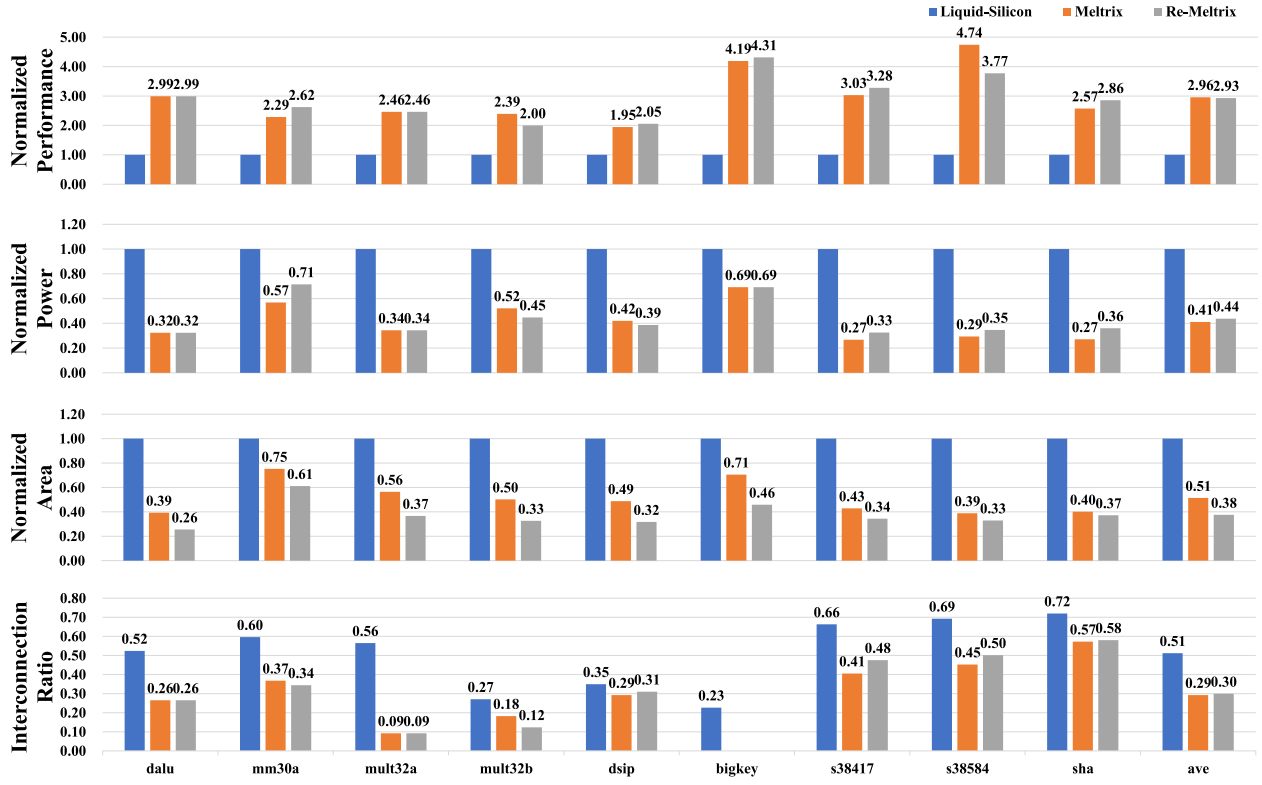


Fig. 11. Logic mode evaluation compared with L-Si and Meltrix.

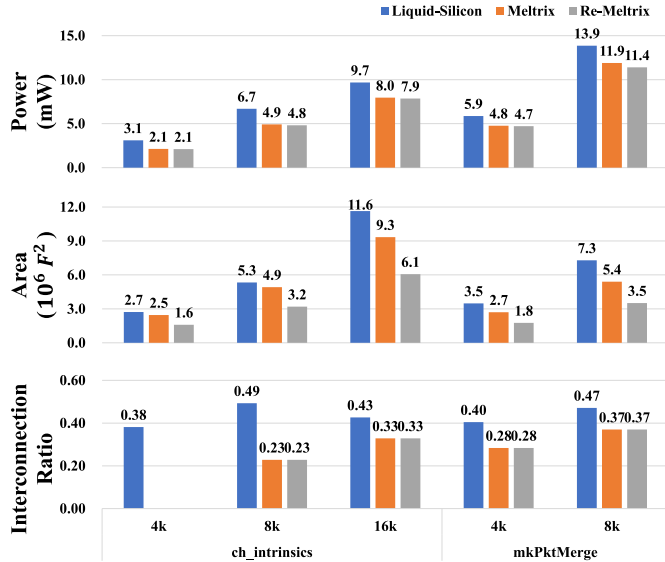


Fig. 12. Memory mode evaluation compared with L-Si and Meltrix.

of the total delay. As a result, the differences between the performance of three architectures of the storage module is limited to no more than 2% across all test samples. Therefore, further discussion on performance in the remaining sections of this test will not be pursued.

The power consumption and area results for all test samples are shown in Fig. 12. The results indicate that all architectures exhibit a linear increase in power consumption and area as the storage capacity increases for the same sample. Across

all test cases, Re-Meltrix demonstrates the lowest power consumption and smallest area compared with L-Si, with power consumption reduced by 17.8% to 32% and area reduced by 40.1% to 51.8%. This suggests that Re-Meltrix not only has superior logic capabilities but also outperforms L-Si in storage efficiency. Regarding routing, L-Si requires three logic mode tiles and one storage mode tile for each storage module, whereas Re-Meltrix needs at most one logic mode tile and one storage mode tile. Additionally, larger logic storage granularity of Re-Meltrix results in a 44% reduction in interconnection ratio compared with L-Si in storage testing, further highlighting the advantage of Re-Meltrix in mitigating global routing challenges. Compared with Meltrix, Re-Meltrix reduces power consumption by 2% and area by 35%. These results demonstrate that Re-Meltrix surpasses Meltrix in all metrics and exhibits enhanced performance through hardware routing optimization and software integration. In the test with *ch_intrinsics* 4k, similar to the *bigkey* in the logic test, Re-Meltrix achieved zero interconnection power consumption, indicating complete decomposition of routing through internal and direct connections within the tile. This example reinforces the advantages of software and hardware architecture design of Re-Meltrix.

D. TCAM Mode Evaluation

We test the TCAM module on L-Si, Meltrix, and Rr-Meltrix using configurations with 1k, 2k, and 4k entries, and the results are shown in Fig. 13. The results indicate that, as the number of entries increases, the performance of all TCAM modules declines due to the increasingly complex logic architecture

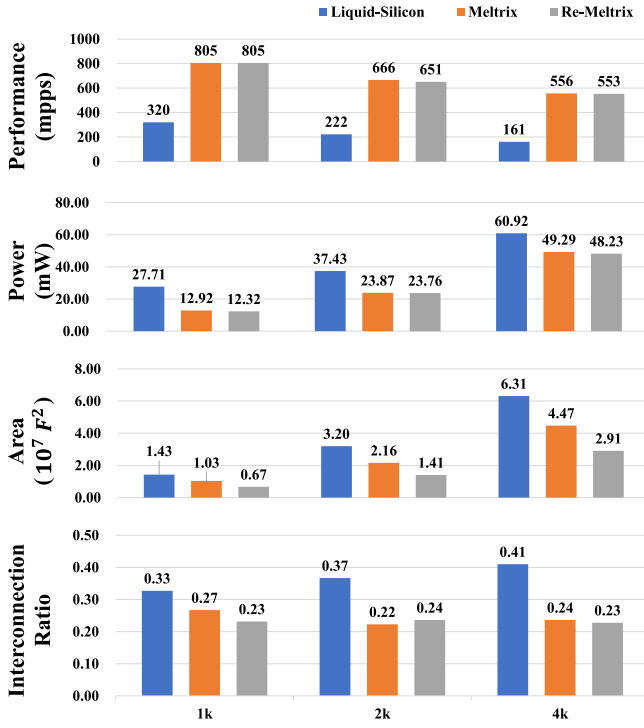


Fig. 13. TCAM evaluation compared with L-Si and Meltrix.

of the priority selector. Despite this, Re-Meltrix maintains a performance advantage of $1.52\times$ to $2.43\times$ over L-Si, with this advantage becoming more pronounced as the number of entries increases. Furthermore, the power consumption reduction of 21% to 56% demonstrates a significant decrease in power consumption compared with L-Si, while Re-Meltrix continues to deliver excellent performance. In terms of area, Re-Meltrix exhibits a 55% reduction compared with L-Si and a 35% advantage over Meltrix, despite a performance decrease of less than 2% and a power consumption increase of less than 5%. This underscores the superiority of the LL control router and retro clustering algorithm of Re-Meltrix compared with those in Meltrix. Lastly, regarding the proportion of interconnection, Re-Meltrix shows an advantage of 29% to 44% over L-Si and a 13% advantage over Meltrix. These results indicate that the use of logic expression mode based on LUT, coarse-grained global routing, and tile internal clustering with LL control routers make it superior to both L-Si and Meltrix in all aspects.

E. Case Study I: Few-Shot Learning

By properly utilizing the different modes of Re-Meltrix, we can implement a complete application on it. In this part, we implement a few-shot learning application based on a binary convolutional neural network (BCNN) using L-Si, Meltrix, and Re-Meltrix, and compare the performance differences across the architectures by measuring latency, power consumption, and area. The specific components of this application are illustrated in Fig. 14, comprising a BCNN and memory. We employ the binary version of the four-layer convolutional network from [38] to execute a 5-way 5-shot few-shot learning task. This BCNN consists of four convolutional layers, where the first two layers use 3×3 filters with 64 channels, and the

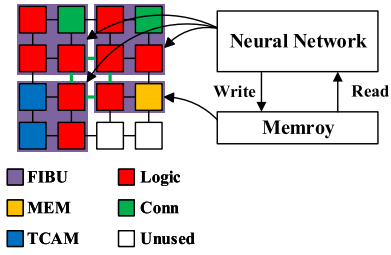


Fig. 14. Illustration of few-shot learning on Re-Meltrix.

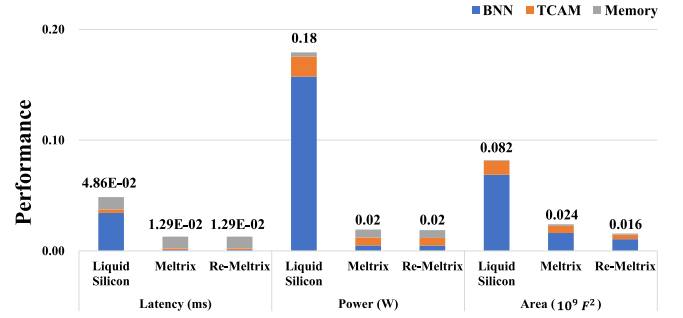


Fig. 15. Latency, power, and area breakdown of Re-Meltrix, Meltrix, and L-Si for a complete few-shot learning application.

last two layers use 3×3 filters with 128 channels. The input data is a 28×28 pixel image from the Omniglot dataset.

The memory is divided into two parts: the first part stores weights, activation values, and other data required for forward computation and backward propagation in the BCNN. The second part utilizes the TCAM encoding method from [39] to efficiently compute L_1 and L_∞ distance metrics in [40], [41], [42]. Specifically, we first use the TCAM mode to identify the nearest neighbor set based on the L_∞ distance metric for the input data, and then use the logic mode to calculate the L_1 -based nearest neighbors within this set, which represent the most likely discriminative results for the input. The method described above has been validated in [42] and shown to achieve acceptable accuracy. It is important to note that the resources used in these methods do not conflict with each other; thus, the BCNN implemented in logic mode, the distance metric calculation in TCAM mode, and the data update in memory mode can be executed in a pipeline, achieving accelerated processing.

The evaluation results are presented in Fig. 15. It is important to note that the delay depicted in the figure represents the total time required for one forward propagation. The results indicate that, compared with L-Si, Re-Meltrix achieves a 3.78-fold improvement in speed while simultaneously reducing power consumption by 86% with lower latency. These metrics surpass the overall average results of previous evaluation, demonstrating that Re-Meltrix is more capable than L-Si in integrating the advantages of multiple modes and handling complex applications more efficiently. Additionally, when compared with Meltrix, Re-Meltrix exhibits a 33% decrease in area while maintaining equivalent delay and power consumption. Combined with previous experiments, the single-tile area reduction rate of Re-Meltrix compared with Meltrix is 34%.

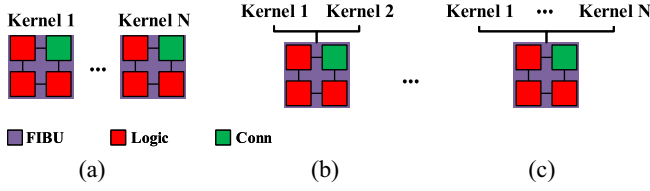


Fig. 16. Different deployment methods on Re-Meltrix (a) Fully parallel. (b) Partially parallel. (c) Fully serial.

This comparison suggests that Re-Meltrix introduces minimal additional tile consumption when dealing with complete complex applications, indicating that the LL control router proposed in this article is highly effective.

F. Case Study II: Deep Neural Network

DNNs are among the most promising applications in modern computing. To evaluate performance and versatility, we deploy DNNs separately on the RRAM-based CNN accelerators, ISAAC[43], and Re-Meltrix, and conduct a comparative analysis. In this experiment, we use VGG-19 [44], a widely adopted convolutional neural network comprising 16 convolutional layers, 3 fully connected layers, and 144 million parameters, as the benchmark. It is important to note that neither ISAAC nor Re-Meltrix is designed for floating-point operations. Therefore, the parameters of VGG-19 in this experiment are represented in a 16-bit fixed-point format. According to [45], this parameter type is sufficient to meet the accuracy requirements of inference tasks.

Due to the high flexibility of Re-Meltrix, VGG-19 can be deployed using multiple methods, each with different performance indicators. Fig. 16 presents several deployment methods. In (a), each kernel in the same layer is assigned to a separate computing module; in (b), every two kernels in the same layer share a computing module; and in (c), all kernels in the same layer share a single computing module. Among these three methods, the delay increases sequentially, while area and power consumption decrease sequentially. In this experiment, we select two representative deployment methods for comparison with ISAAC: 1) the latency-efficient method (Re-Meltrix-L) and 2) the balanced method (Re-Meltrix-B), corresponding to (a) and (b) in Fig. 16.

The experimental results are presented in Table II, which reports the inference delay, power consumption, area, and power-delay product (PDP) for each deployment. The results indicate that Re-Meltrix-L achieves a 7.48-fold increase in inference speed compared with ISAAC while reducing the area by 71.76%, power consumption by 25.74% and PDP by 90.1%. Similarly, Re-Meltrix-B demonstrates substantial improvements across various metrics compared with ISAAC, including a 3.76-fold increase in inference speed, a 65.23% reduction in power consumption, and a 85.89% reduction in area. When comparing Re-Meltrix-L and Re-Meltrix-B, the PDP of Re-Meltrix-B decreases by 6.33% relative to Re-Meltrix-L. This reduction is primarily attributed to the latency-prioritized deployment in Re-Meltrix-L, which tends to reduce multiplier reuse within the same layer, leading to

TABLE II
INFERENCE PERFORMANCE RESULTS

Architecture	Delay (ms)	Power (W)	Area ($10^9 F^2$)	PDP
Re-Meltrix-L	1.07	48.86	23.55	52.12
Re-Meltrix-B	2.13	22.88	11.77	48.82
ISAAC	8.00	65.80	83.40	526.40

the deployment of more tiles and increased interconnection complexity, thus incurring higher costs to reduce latency. These results clearly demonstrate that, for DNNs, Re-Meltrix not only delivers strong performance compared with dedicated accelerators but also offers exceptional flexibility and universality.

VI. CONCLUSION

We have developed a reconfigurable PIM architecture by leveraging the switching characteristics of RRAM, multimode fusion signal paths, and co-design functional synthesis. The experimental results presented in this article indicate that our Re-Meltrix architecture significantly improves performance in logic, storage, and TCAM modes compared with L-Si, effectively addressing the routing overhead and logic mapping challenges in general computing reconfigurable PIM architectures.

More importantly, the evaluation results of Re-Meltrix demonstrate that as applications become increasingly complex, it is essential to increase logic granularity and achieve flexible storage deployment within a reconfigurable architecture. Additionally, our experimental findings indicate that fully utilizing local and intratile interconnectivity can significantly reduce the complexity of global routing, thereby minimizing the excessive use of hardware resources for routing devices.

REFERENCES

- [1] X. Chen, J. Zhu, J. Jiang, and C.-Y. Tsui, "CompRRAE: RRAM-based convolutional neural network accelerator with reduced computations through a runtime activation estimation," in *Proc. 24th Asia South Pac. Design Autom. Conf. (ASP-DAC)*, 2019, pp. 1–7.
- [2] Z. Zhu et al., "Mixed size crossbar based RRAM CNN accelerator with overlapped mapping method," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2018, pp. 1–8.
- [3] A. Dorzhigulov, S. Mishra, and V. Saxena, "Hybrid CMOS-RRAM spiking CNNs with time-domain max-pooling and integrator re-use," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2022, pp. 1828–1832.
- [4] S. Bavikadi, P. R. Sutradhar, A. Ganguly, and S. M. P. Dinakarrao, "Heterogeneous multi-functional look-up-table-based processing-in-memory architecture for deep learning acceleration," in *Proc. 24th Int. Symp. Qual. Electron. Design (ISQED)*, 2023, pp. 1–8.
- [5] P. R. Sutradhar et al., "Look-up-table based processing-in-memory architecture with programmable precision-scaling for deep learning applications," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 2, pp. 263–275, Feb. 2022.
- [6] J. Sun, F. Liu, Y. Zhang, L. Jiang, and R. Yang, "RTSA: An RRAM-TCAM based in-memory-search accelerator for sub-100 μ s collision detection," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, 2024, pp. 1–2.
- [7] Y. Liao et al., "Parasitic resistance effect analysis in RRAM-based TCAM for memory augmented neural networks," in *Proc. IEEE Int. Memory Workshop (IMW)*, 2020, pp. 1–4.
- [8] K. Pan, A. M. Tosson, N. Wang, N. Y. Zhou, and L. Wei, "A novel cascable TCAM using RRAM and current race scheme for high-speed energy-efficient applications," *IEEE Trans. Nanotechnol.*, vol. 22, pp. 214–221, Apr. 2023.

- [9] Z. Chen, H. Zhou, and J. Gu, "R-accelerator: An RRAM-based CGRA accelerator with logic contraction," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 11, pp. 2655–2667, Nov. 2019.
- [10] Y. Zhai, B. Li, B. Yan, and J. Wang, "STAR: An efficient Softmax engine for attention model with RRAM crossbar," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, 2023, pp. 1–2.
- [11] Y. He, Y. Huang, J. Yue, W. Sun, L. Zhang, and Y. Liu, "C-RRAM: A fully input parallel charge-domain RRAM-based computing-in-memory design with high tolerance for RRAM variations," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2022, pp. 3279–3283.
- [12] S. Bavikadi, P. R. Sutradhar, M. A. Indovina, A. Ganguly, and S. M. P. Dinakarrao, "ReApprox-PIM: Reconfigurable approximate lookup-table (LUT)-based processing-in-memory (PIM) machine learning accelerator," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 8, pp. 2288–2300, Aug. 2024.
- [13] L. Chang, Z. Wang, Y. Zhang, and W. Zhao, "Reconfigurable processing in memory architecture based on spin orbit torque," in *Proc. IEEE/ACM Int. Symp. Nanoscale Archit. (NANOARCH)*, 2017, pp. 95–96.
- [14] S. Bavikadi, P. R. Sutradhar, A. Ganguly, and S. M. P. Dinakarrao, "Reconfigurable processing-in-memory architecture for data intensive applications," in *Proc. 37th Int. Conf. VLSI Design Proc. 23rd Int. Conf. Embed. Syst. (VLSID)*, 2024, pp. 222–227.
- [15] X. Tang, E. Giacomini, P. Cadareanu, G. Gore, and P.-E. Gaillardon, "A RRAM-based FPGA for energy-efficient edge computing," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, 2020, pp. 144-a–144-f.
- [16] N. C. Dao and D. Koch, "Memristor-based pass gate for FPGA programmable routing switch," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2021, pp. 1–5.
- [17] J. P. Cardoso de Lima, M. Brandalero, and L. Carro, "Endurance-aware RRAM-based reconfigurable architecture using TCAM arrays," in *Proc. 30th Int. Conf. Field-Program. Logic Appl. (FPL)*, 2020, pp. 40–46.
- [18] S. Park, J. Kim, and S. Yoon, "Energy-aware placement for SRAM-NVM hybrid FPGAs," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, 2020, pp. 858–863.
- [19] J. D. Ferreira et al., "pLUTO: Enabling massively parallel computation in DRAM via lookup tables," in *Proc. 55th IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2022, pp. 900–919.
- [20] Q. Deng, Y. Zhang, M. Zhang, and J. Yang, "LAcc: Exploiting lookup table-based fast and accurate vector multiplication in DRAM-based CNN accelerator," in *Proc. 56th ACM/IEEE Design Autom. Conf. (DAC)*, 2019, pp. 1–6.
- [21] A. D. Gunter and S. Wilton, "Towards a machine learning approach to predicting the difficulty of FPGA routing problems," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays*, 2023, p. 231.
- [22] Y. Zha, E. Nowak, and J. Li, "Liquid silicon: A nonvolatile fully programmable processing-in-memory processor with monolithically integrated ReRAM," *IEEE J. Solid-State Circuits*, vol. 55, no. 4, pp. 908–919, Apr. 2020.
- [23] B. Long, L. Shen, X. Zhang, Y. Han, X.-H. Sun, and X. Chen, "Meltrix: A RRAM-based polymorphic architecture enhanced by function synthesis," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, 2023, pp. 1–9.
- [24] B. Govoreanu et al., "Performance and reliability of ultra-thin HfO₂-based RRAM (UTO-RRAM)," in *Proc. 5th IEEE Int. Memory Workshop*, 2013, pp. 48–51.
- [25] A. Grossi et al., "Fundamental variability limits of filament-based RRAM," in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, 2016, pp. 4.7.1–4.7.4.
- [26] A. Bende et al., "Experimental validation of memristor-aided logic using 1T1R TaOx RRAM crossbar array," in *Proc. 37th Int. Conf. VLSI Design Proc. 23rd Int. Conf. Embed. Syst. (VLSID)*, 2024, pp. 565–570.
- [27] W.-C. Huang et al., "A functional novel logic for max/min computing in one-transistor-one-resistor devices with resistive random access memory (RRAM)," *IEEE Trans. Electron Devices*, vol. 69, no. 4, pp. 1811–1815, Apr. 2022.
- [28] *IEEE Standard for VHDL Language Reference Manual*, IEEE Standard 1076-2019, 2019.
- [29] *IEEE Standard for System Verilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Standard 1800-2017 (Revision of IEEE Std 1800-2012), 2018.
- [30] P. Jamieson, K. B. Kent, F. Gharibian, and L. Shannon, "Odin II—an open-source verilog HDL synthesis tool for CAD research," in *Proc. 18th IEEE Annu. Int. Symp. Field-Program. Custom Comput. Mach.*, 2010, pp. 149–156.
- [31] K. E. Murray, O. Petelin, S. Zhong, J. M. Wang, and V. Betz, "VTR 8: High-performance CAD and customizable FPGA architecture Modelling," *ACM Trans. Reconfig. Technol. Syst.*, vol. 13, no. 2, pp. 1–55, 2020.
- [32] R. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *Proc. 22nd Int. Conf. Comput. Aided Verificat.*, 2010, pp. 24–40.
- [33] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [34] Y. Zha and J. Li, "Revisiting pathfinder routing algorithm," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, 2022, pp. 24–34.
- [35] "METIS, Karypis Lab." 2023. [Online]. Available: <https://github.com/KarypisLab/METIS>
- [36] S. Yang, *Logic Synthesis and Optimization Benchmarks User Guide: Version 3.0*, Microelectron. Center North Carolina, Greensboro, NC, USA, 1991.
- [37] B. Hajri, H. Aziza, M. M. Mansour, and A. Chehab, "RRAM device models: A comparative analysis with experimental validation," *IEEE Access*, vol. 7, pp. 168963–168980, 2019.
- [38] L. Kaiser, O. Nachum, A. Roy, and S. Bengio, "Learning to remember rare events," in *Proc. Int. Conf. Learn. Represent.*, Mar. 2017, pp. 1–10.
- [39] A. Bremner-Barr, Y. Harchol, D. Hay, and Y. Hel-Or, "Encoding short ranges in TCAM without expansion," in *Proc. 28th ACM Symp. Parallelism Algorithms Archit.*, Jul. 2016, pp. 35–46.
- [40] A. F. Laguna, X. Yin, D. Reis, M. Niemier, and X. S. Hu, "Ferroelectric FET based in-memory computing for few-shot learning," in *Proc. Great Lakes Symp. VLSI*, May 2019, pp. 373–378.
- [41] D. Reis, A. F. Laguna, M. Niemier, and X. S. Hu, "Attention-in-memory for few-shot learning with configurable ferroelectric FET arrays," in *Proc. 26th Asia South Pac. Design Autom. Conf.*, Jan. 2021, pp. 49–54.
- [42] A. F. Laguna, M. Niemier, and X. S. Hu, "Design of hardware-friendly memory enhanced neural networks," in *Proc. Design, Autom., Test Europe (DATE)*, Mar. 2019, pp. 1583–1586.
- [43] A. Shafiee et al., "ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, 2016, pp. 14–26.
- [44] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015, *arXiv:1409.1556*.
- [45] D. Lin, S. Talathi, and S. Annapureddy, "Fixed point quantization of deep convolutional networks," in *Proc. 33rd Int. Conf. Mach. Learn.*, New York, New York, USA, Jun. 2016, pp. 2849–2858.



Boyu Long received the B.E. degree from the Department of Electronic Engineering, Tsinghua University, Beijing, China, in 2021. He is currently pursuing the Ph.D. degree with the University of Chinese Academy of Sciences, Beijing.

His research interests include electronic design automation and computing-in-memory architecture design.



Yinhe Han (Senior Member, IEEE) received the M.S. and Ph.D. degrees in computer science from the Institute of Computing Technology (ICT), Chinese Academy of Sciences (CAS), Beijing, China, in 2003 and 2006, respectively.

He is currently a Professor with ICT, CAS. His research interests include microprocessor design, integrated circuit design, and computer architecture.



Xian-He Sun (Life Fellow, IEEE) received the B.S. degree in mathematics from Beijing Normal University, Beijing, China, in 1982, and the M.S. and Ph.D. degrees in computer science from Michigan State University, East Lansing, MI, USA, in 1987 and 1990, respectively.

He is a Distinguished Professor with the Department of Computer Science, Illinois Institute of Technology, Chicago, IL, USA. His research interests include parallel and distributed processing, memory, and I/O systems.



Xiaoming Chen (Member, IEEE) received the B.S. and Ph.D. degrees in electronic engineering from Tsinghua University, Beijing, China, in 2009 and 2014, respectively.

He is currently a Professor with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing. His current research interest is focused on design automation for integrated circuits and computing-in-memory architectures.